

AI-Furqan - The Criterion

Current architecture overview for the axiom-anchored neuro-symbolic reasoning engine. Generated from the README architecture sweep on 2026-06-21. Markdown READMEs and live code remain the source of truth.

Core Principle

```
The code routes and scores.  
The LLM extracts, reasons, and explains.  
Z3 checks formal consistency.  
Elasticsearch retrieves verified knowledge and stores verdicts.  
Humans review and correct.
```

Canonical Gates

- Source-Integrity: raw truth must be preserved without reduction.
- Structural-Consistency: claims must explain causality without luck.
- Mediation-Zeroing: human cognition is finite, not authoritative.
- Origin-Aware: truth must trace to a transcendent source.
- Implemented extension: Origin-Preservation is carried in verdict models, API schemas, and ES storage.

Layered Architecture

Layer	Current components
Client surfaces	CLI, FastAPI REST API, furqan-raas MCP, furqan-memory MCP
Security envelope	API keys, roles, rate limits, request limits, security headers, PromptGuard, IntegrityVerifier, OutputValidator, AdapterSandbox, AuditLogger
Orchestration	FastAPI routers and Orchestrator connect engine, KB, stores, Z3, and LLMs. Cross-layer access belongs here.
Reasoning engine	Axioms, intent detection, Scan -> Mirror -> Verdict -> Self-Correction, dual perspective, gates, chains, deterministic scoring, Z3, Tafsir RAG
Knowledge and retrieval	ES-backed Quran, Hadith, graph, lessons, Tafsir KB tools, tokenizer, ingestion, proposed-edge review
Data and learning	data_archive for source/generated data, training/pipeline for graph/history outputs, ES runtime persistence, local-only Memory MCP

Boundary Rule

Only orchestration should know about every layer. The engine receives context as text and returns verdict objects; the KB returns retrieval context; stores persist and retrieve verdict/feedback documents.

Runtime Surfaces

Surface	Entry point	Responsibility
CLI	al-furqan	Local command-line access to the core package
REST API	uvicorn al_furqan.api.app:app	Evaluation, grounded evaluation, Criterion tests, verdict review, stats, health
RaaS MCP	python -m furqan_raas.mcp_server	JSON-RPC/MCP wrapper for reasoning tools
Memory MCP	python -m furqan_memory.mcp_server	Local agent memory, recall, pattern recognition, feedback

Primary REST Endpoints

Endpoint	Purpose
POST /api/v1/evaluate	Run the standard evaluation pipeline
POST /api/v1/evaluate/grounded	Evaluate with retrieved precedent/context
POST /api/v1/criterion-test	Evaluate a named framework
GET /api/v1/verdicts/search	Search verdicts
POST /api/v1/verdicts/{id}/review	Approve, reject, or correct a verdict
GET /api/v1/stats and /api/v1/health	Store statistics and component health

Storage and Search

Logical name	Default ES index	Purpose
quran	furqan_quran	Quran verses, Arabic/English text, metadata, embeddings
hadith	furqan_hadith	Hadith records, grading metadata, embeddings
graph	furqan_graph	Knowledge graph edges and provenance
lessons	furqan_lessons	Enriched lesson chapters and linked evidence
verdicts	furqan_verdicts	Evaluation results and precedent retrieval
feedback	furqan_feedback	Human feedback and corrections

Training/Staging Indices

Plan key	ES index	Source JSONL
graph	furqan_graph_edges	data_archive/training/quran_graph.jsonl
history	furqan_history_events	data_archive/training/human_history.jsonl
testing_talk_about_history	furqan_testing_talk_about_history	data_archive/training/testing/model_testing_how_people_talk_about_history.jsonl

Repository Map

Path	Responsibility
src/al_furqan/api	FastAPI app, routers, schemas, orchestrator
src/al_furqan/auth	API key auth, roles, rate limiting, security middleware
src/al_furqan/engine	Axioms, gates, chains, symbolic verifier, security, Tafsir RAG
src/al_furqan/kb	ES collections, graph schema, ingestion, Tafsir KB tools
src/al_furqan/store	ES verdict and feedback stores
src/al_furqan/tokenizer	Quran tokenizer and QAC-aware morphology
furqan-raas	MCP reasoning server wrapper
furqan-memory	Local MCP memory server
data_archive	Archived corpora and generated training JSONL
training	Graph/history extraction and staging pipelines

Current Tree Snapshot

Area	Count
Core Python modules under src/al_furqan	110
Core test modules under tests	42
RaaS test modules	1
Memory test modules	4
Test functions across core/RaaS/Memory	684

Operational Commands

```
pip install -e ".[dev]"
pytest -m "not slow"
ruff check src/ tests/
uvicorn al_furqan.api.app:app --reload
docker compose up elasticsearch redis
python -m al_furqan.kb.es.setup_indices --test
python -m al_furqan.kb.es.migrate_data --data-dir data_archive/backup_for_stored_data --verify
python -m training.pipeline.staging.index_training_data --dry-run
```

Operational note: docker-compose.yml defines an es-seed service, but the current Dockerfile.es-seed references a legacy data/ directory and a scripts/seed_es.sh file that is not present in this checkout. Use explicit setup/migration commands until reconciled.

Documentation and Maintenance

Document	Role
docs/active_docs/AL-FURQAN-ARCHITECTURE-v3.0.md	Baseline architecture plus post-v3 update notes
docs/active_docs/PROJECT-STATUS.md	Latest project status and decisions
docs/active_docs/ELASTICSEARCH-MIGRATION.md	ES migration rationale and schemas
docs/active_docs/QURAN-TOKENIZER-v1.0.md	Current tokenizer architecture
docs/active_docs/RAG-IMPLEMENTATION-PLAN-v1.0.md	Engine-guided RAG design
docs/active_docs/FURQAN-AXIOM-SECURITY-POLICY-v1.0.md	Security model
furqan-raas/README.md	Current RaaS MCP boundary
furqan-memory/README.md	Current Memory MCP boundary
data_archive/README.md	Current archive/training data layout

Architecture Rules

- Do not normalize, strip, transliterate, or stem Arabic text outside the tokenizer/analyzer paths.
- Do not edit axiom definitions without updating anchored SHA-256 expectations.
- Route LLM calls through providers/ or injected callables; avoid raw provider HTTP calls in feature code.
- Keep layer boundaries: orchestration connects engine, KB, stores, and security.
- Treat furqan-memory as local client memory, not core engine storage.
- Treat generated JSONL as rebuildable pipeline output.

Tech Stack

Layer	Technology
Language/API	Python 3.10+, FastAPI, uvicorn
Providers	Anthropic, DashScope/Qwen, Ollama, OpenAI-compatible
Formal verification	Z3 SMT solver
Runtime storage/search	Elasticsearch 8.x with custom Arabic analyzer
Local memory skill	SQLite + ChromaDB
Optional extras	sentence-transformers, faster-whisper, Neo4j loaders